

AUTOMATED PCB DEFECT DETECTION USING COMPUTER VISION

Matt Miller
Computer Vision
Dr. Subil Abraham
March 8, 2026

Problem Statement

PCB manufacturing requires high-throughput defect inspection where missed defects risk costly product failures (\$500/board) while false positives waste rework time (\$50/board). This project builds a reproducible pipeline detecting six defect types (missing hole, mouse bite, open circuit, short, spur, and spurious copper) comparing classical template matching, YOLOv8 object detection, and a two-stage YOLOv8+ResNet pipeline, with ONNX export for deployment.

Datasets and Preprocessing

The Kaggle PCB Defects dataset provides 1,386 images (693 originals + 693 pre-rotated) with 2,953 bounding box annotations across 6 balanced classes (imbalance ratio 1.04x). Pascal VOC XML annotations were converted to YOLO format and split 70/15/15 stratified (485/104/104). DeepPCB provides 1,500 paired template/defective images for template matching evaluation only. Images are letterbox-resized to 640x640. Augmentation: mosaic, flips, 90° rotation, HSV jitter, scale variation.

Model Architectures

Template Matching: Align defect-free template to test image using ORB feature matching with RANSAC homography. Subtract pixel-by-pixel, apply Otsu thresholding, extract bounding boxes from contours.

YOLOv8: Single-pass anchor-free detector - processes the entire image once, outputting all bounding boxes and class predictions simultaneously. Transfer learning from COCO (80 classes) to our 6 defect types. Loss optimizes box accuracy (CIoU), classification (BCE), and boundary precision (DFL) jointly.

ResNet-18: ImageNet-pretrained, FC layer replaced (512→6). Two-stage: YOLOv8 detects → ResNet classifies crops.

Training Configuration

I used YOLOv8n (nano, 3.2M parameters) - the smallest variant, chosen for fast iteration on a limited dataset (693 images). Larger variants (YOLOv8s at 11.2M, YOLOv8m at 25.9M) offer higher capacity but risk overfitting on small datasets.

Training ran for 70 epochs with early stopping (patience=10), best model saved at epoch 60. All three loss components (box, classification, DFL) converged smoothly with no signs of overfitting. PCB-specific augmentation was applied: vertical flip and 90° rotation enabled (off by default) since boards have no canonical orientation, and mosaic augmentation aids small defect detection by combining four training images.

	YOLOv8n	ResNet-18
Epochs	70 (early stopped at 60)	30
Optimizer	AdamW, lr=0.01	Adam, lr=1e-4
Input size / Batch	640x640 / 16	224x224 (GT crops) / 32
Frozen layers	-	conv1-layer3

Results and Evaluation

Approach	mAP@0.5	Precision	Recall	F1	Inference (ms)
Template Matching	N/A	0.002	0.012	0.004	-
YOLOv8	0.804	0.821	0.747	0.782	35.0
ResNet-18 (GT crops)	N/A	1.000	1.000	1.000	-
YOLOv8+ResNet (two-stage)	0.804	0.821	0.747	0.782	~79

The 10x cost asymmetry between missed defects and false alarms means 0.747 recall is the primary metric to improve; lowering the confidence threshold would trade precision for recall, better matching the business objective.

Template matching failed (F1=0.004) due to false positives from alignment artifacts - YOLOv8 achieved 195x higher F1. Per-class: missing_hole (0.962 AP) near-perfect; spur (0.660) and mouse_bite (0.672) hardest. Two-stage showed 98.1% agreement with YOLOv8 across 426 detections but added 44ms latency - single-pass recommended.

ONNX INT8 quantization: 12MB→3.5MB (3.5x compression), mAP drop 0.48%, recall improved 1.1%. Inference: PyTorch 35ms → ONNX FP32 28ms → INT8 31ms. All quality gates passed.

Challenges and Solutions

Data pipeline: Two annotation formats (VOC XML, custom TXT) with different class orderings required dedicated converters. High-resolution images (2778x2138) resized to 640x640 risk losing small defects; letterbox padding and multi-scale augmentation mitigate this.

Model limitations: Spur and mouse_bite are small and subtle (AP ~0.66) — path forward includes larger model variants and lower confidence threshold favoring recall. All images derive from one PCB layout; production requires per-board fine-tuning (~100-200 labeled images via transfer learning).